

Source Code of App: camera

admin.py

```
from django.contrib import admin

# Register your models here.
```

apps.py

```
from django.apps import AppConfig

class CameraConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "camera"
```

calibrate_func.py

```
import cv2
import numpy as np

def calculate_iou(bbox1, bbox2):
    x1_1, y1_1, x2_1, y2_1 = bbox1 # สกัด position ของ แกนx และ y ของ bounding box person
    x1_2, y1_2, x2_2, y2_2 = bbox2 # สกัด position ของ แกนx และ y ของ bounding box chair

    x_max_left = max(x1_1, x1_2) #หาจุดตัดของขอบซ้าย
    y_max_top = max(y1_1, y1_2) #หาจุดตัดของขอบวา
    x_min_right = min(x2_1, x2_2) #หาจุดตัดของขอบบน
    y_min_bottom = min(y2_1, y2_2) #หาจุดตัดของขอบล่าง

    # คำนวณหาจุดของ ความกว้าง และ ความสูง ของพื้นที่ทับซ้อน
    inter_width = max(0, x_min_right - x_max_left)
    inter_height = max(0, y_min_bottom - y_max_top)

    # คำนวณหาขนาดของพื้นที่ที่ทับซ้อนกัน
    intersection_area = inter_width * inter_height

    # คำนวณหาจุดที่ สองกล่อง Unionกัน
    area1 = (x2_1 - x1_1) * (y2_1 - y1_1)
    area2 = (x2_2 - x1_2) * (y2_2 - y1_2)

    # คำนวณหาพื้นที่ที่สองกล่อง Union กัน
    union_area = area1 + area2 - intersection_area

    # คำนวณหาค่าIoU
    iou = intersection_area / union_area if union_area > 0 else 0

    return iou

def check_brightness(frame, threshold=50):
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    avg_brightness = np.mean(gray_frame)
    return avg_brightness, int(avg_brightness < threshold) # แปลงเป็น Integer

def is_sit_chair(user_bbox, chair_bbox, threshold=0.3):
    iou = calculate_iou(user_bbox, chair_bbox) #คำนวณหาค่าIoUของ bbox_user และ bbox_chair
    return iou >= threshold
```

models.py

```
from django.db import models

# Create your models here.
```

urls.py

```
from django.urls import path
from . import views
app_name = 'camera'
urlpatterns = [
    path('Detection/', views.CameraDetectionView.as_view(), name='camera_detection'),
    path('process_image/', views.process_image, name='process_image'),
    path('DetectHub/' , views.Selection_hubView.as_view(), name='DetectHub'),
    path('posture-detection/', views.posture_detection, name='posture-detection'),
    path('Estimation/', views.Detection_View.as_view(), name='estimation'),
    path('Upload-Image/', views.Image_View.as_view(), name='Image-upload'),
    path('save-detection-result/', views.save_detection_result, name='save_detection_result'),
]
```

views.py

```
from django.http import JsonResponse
import base64
from io import BytesIO
from PIL import Image
import numpy as np
```

```

import cv2
import json
from django.views.generic import TemplateView
from ultralytics import YOLO
from camera.calibrate_func import *
import mediapipe as mp
from .utils.posture_analysis import calculate_angles , calculate_score
from base.models import PostureDetection , UserUsageHistory
from django.utils import timezone
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.decorators import login_required
from django.contrib.auth.mixins import LoginRequiredMixin

class CameraDetectionView(LoginRequiredMixin,TemplateView):
    template_name = 'camera.html'

class Selection_hubView(LoginRequiredMixin,TemplateView):
    template_name = 'selection_hub.html'

class Detection_View(LoginRequiredMixin,TemplateView):
    template_name = 'detection.html'

class Image_View(LoginRequiredMixin,TemplateView):
    template_name = 'upload_image.html'

# โมเดล โอลิมปิก YOLOv8

model = YOLO('yolov8n.pt')
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()

def is_full_body(pose_landmarks):
    left_knee = pose_landmarks.landmark[mp_pose.PoseLandmark.LEFT_KNEE]
    right_knee = pose_landmarks.landmark[mp_pose.PoseLandmark.RIGHT_KNEE]

    if left_knee.visibility > 0.2 and right_knee.visibility > 0.2:
        return True
    return False

def process_image(request):
    if request.method == 'POST':
        try:
            # รับข้อมูลภาพจาก request
            data = json.loads(request.body)
            image_data = data.get('image')

            if not image_data:
                return JsonResponse({'error': 'No image data provided'}, status=400)

            try:
                # แปลง Base64 เป็น NumPy array
                img_data = base64.b64decode(image_data.split(',')[1]) # ลบ prefix base64
                img = Image.open(BytesIO(img_data))

                # แปลง RGBA เป็น RGB (ถ้าจำเป็น)
                if img.mode == 'RGBA':
                    img = img.convert('RGB')

                img = np.array(img)

                # แปลง RGB เป็น BGR (OpenCV ใช้ BGR เป็นค่าเริ่มต้น)
                img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
            except Exception as e:
                return JsonResponse({'error': f"Error decoding image: {str(e)}"}, status=500)

            # ตรวจสอบขนาดของภาพ
            try:
                height, width, _ = img_bgr.shape
                print(f"Image size: {width}x{height}")
            except Exception as e:
                return JsonResponse({'error': f"Error processing image dimensions: {str(e)}"}, status=500)

            # ตรวจสอบความสว่าง
            try:
                avg_brightness, is_low_brightness = check_brightness(img_bgr)
            except Exception as e:
                return JsonResponse({'error': f"Error checking brightness: {str(e)}"}, status=500)

            # ตรวจสอบวัตถุด้วย YOLOv8
            try:
                results = model(img_bgr) # ตรวจสอบวัตถุ
                detections = []
                person_bbox = None # เก็บ bounding box ของ person
                chair_bbox = None # เก็บ bounding box ของ chair

                for r in results:
                    for box in r.bboxes:
                        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist()) # แปลง Tensor เป็น List และ Map เป็น int
                        class_id = int(box.cls) # class id
                        if class_id == 0 or class_id == 56:
                            class_name = model.names[class_id] # ชื่อ class
                            confidence = float(box.conf[0]) * 100 # ความมั่นใจ (%) แปลง Tensor เป็น float
                            bbox = [x1, y1, x2, y2] # Bounding box

                            # เพิ่ม bounding box ลงใน detections
                            detections.append({

```

```

        'class_name': class_name,
        'confidence': confidence,
        'bbox': bbox
    })

    # แยก Bounding Box สำหรับ person และ chair
    if class_id == 0: # Person
        person_bbox = bbox
    elif class_id == 56: # Chair
        chair_bbox = bbox
except Exception as e:
    return JsonResponse({'error': f"Error in YOLOv8 detection: {str(e)}"}, status=500)

# ตรวจสอบว่าบุคคลนั้นนั่งเก้าอี้หรือไม่
is_sitting = False
if person_bbox and chair_bbox:
    is_sitting = is_sit_chair(person_bbox, chair_bbox)
try:
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    results_pose = pose.process(img_rgb)
    print(results_pose)
    is_full_body_detected = False

    if results_pose.pose_landmarks:
        is_full_body_detected = is_full_body(results_pose.pose_landmarks)
except Exception as e:
    return JsonResponse({'error': f"Error in MediaPipe Pose detection: {str(e)}"}, status=500)

# ส่งผลลัพธ์กลับ
return JsonResponse({
    'message': 'Image processed successfully',
    'width': width,
    'height': height,
    'avg_brightness': avg_brightness,
    'is_low_brightness': int(is_low_brightness), # แปลงเป็น 1 (True) หรือ 0 (False)
    'is_full_body_detected': is_full_body_detected,
    'detections': detections,
    'is_sitting': is_sitting # True หากบุคคลนั้นนั่งเก้าอี้
})
except Exception as e:
    return JsonResponse({'error': f"Unexpected error: {str(e)}"}, status=500)

```

```

@csrf_exempt
@login_required
def posture_detection(request):
    # Handle non-POST requests
    if request.method != 'POST':
        return JsonResponse({'error': 'เฉพาะ POST requests เท่านั้น'}, status=405)

    try:
        data = json.loads(request.body)
        image_data = data.get('image')
        detect_type = data.get('detect_type', 'Photo Detection')

        if image_data is None:
            return JsonResponse({'error': "ไม่พบข้อมูลรูปภาพ"}, status=400)

        if detect_type not in ['Photo Detection', 'Side-part Detection']:
            return JsonResponse({'error': 'ประเภทการตรวจจับไม่ถูกต้อง.'}, status=400)

        # แปลงรูปจาก base64
        img_data = base64.b64decode(image_data.split(',')[1])
        img = Image.open(BytesIO(img_data))
        img = np.array(img)

        # ประมวลผลภาพทางด้วย mediapipe
        mp_pose = mp.solutions.pose
        with mp_pose.Pose(static_image_mode=True) as pose:
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            results_pose = pose.process(img_rgb)

            if not results_pose.pose_landmarks:
                return JsonResponse({'error': 'ไม่พบจุดสังเกตท่าทาง'}, status=400)

            angles = calculate_angles(results_pose.pose_landmarks)
            score, feedback = calculate_score(angles)

        # กรณี Photo Detection บันทึกข้อมูลลงฐานข้อมูลทันที
        if detect_type == 'Photo Detection':
            posture_detection_instance = PostureDetection.objects.create(
                user=request.user,
                score=score
            )

            UserUsageHistory.objects.create(
                posture_detection=posture_detection_instance,
                detect_type='Photo Detection'
            )

        return JsonResponse({
            "message": "ตรวจจับท่าทางสำเร็จ",
            "angles": angles,
            "score": score,
            "feedback": feedback,
            "posture_valid": score
        })
    
```

```

except Exception as e:
    return JsonResponse({"error": str(e)}, status=500)

@csrf_exempt
@login_required
def save_detection_result(request):
    # ฟังก์ชันนี้เรียกใช้ตอนที่ถูกกด Stop Detect ในกรณี Continuous Detection
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            score = data.get('score')
            detection_duration = data.get('detection_duration')

            if score is None:
                return JsonResponse({'error': 'คะแนนเป็นข้อมูลที่จำเป็น.'}, status=400)

            if detection_duration is None:
                return JsonResponse({'error': 'จำเป็นต้องมีข้อมูลระยะเวลาในการตรวจจับ.'}, status=400)

            posture_detection_instance = PostureDetection.objects.create(
                user=request.user,
                score=score
            )

            UserUsageHistory.objects.create(
                posture_detection=posture_detection_instance,
                detect_type='Side-part Detection',
                detection_time=detection_duration
            )

            return JsonResponse({"message": "บันทึกผลการตรวจจับเรียบร้อยแล้ว."})

        except Exception as e:
            return JsonResponse({"error": str(e)}, status=500)

```

__init__.py

__init__.py

camera.html

```

{% extends "base.html" %}
{% block main %}

```

Camera Detection



เลือกกล้อง... ▼

กำลังถ่ายทอดสด

Capture

>
ไปหน้าถัดไป

ตรวจสอบความสว่าง



นั่งอยู่บนเก้าอี้ไหม



มองเห็นเต็มตัวไหม



ยืนยันการดำเนินการ

กล่องของคุณยังไม่พร้อมดีนักในการทำงาน คุณต้องการดำเนินการต่อหรือไม่?

ยกเลิก

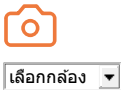
ยืนยัน

```
{% endblock %}
```

detection.html

```
{% extends "base.html" %}
{% block main %}
```

การตรวจจับท่าทาง



กล้องกำลังใช้งาน

▶

เริ่มตรวจจับ

□

หยุดตรวจจับ

00

ชั่วโมง

:

00

นาที่

:

00

วินาที

0%

คะแนน

คำแนะนำล่าสุด

ผลการตรวจจับทำทาง

คำแนะนำจากระบบ

```
{% endblock %}
```

selection_hub.html

```
{% extends "base.html" %}
{% block main %}
```

ตรวจจับทำน้

เลือกโหมดการตรวจจับที่คุณต้องการ



Photo Detection

ตรวจจับท่าทางการนั่งจากภาพถ่าย เพื่อวิเคราะห์ทำน้ของคุณ

[เริ่มต้นใช้งาน](#)

>



Real-time Detection

ตรวจจับท่าทางนั่งแบบเรียลไทม์ เพื่อติดตามท่านั่งของคุณ

[เริ่มต้นใช้งาน](#)

>

```
{% endblock %}
```

upload_image.html

```
{% extends "base.html" %}
{% block main %}
```

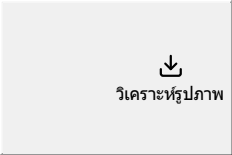
อัปโหลดรูปภาพ

Choose File

No file selected

คลิกเพื่ออัปโหลดรูปภาพ

หรือลากและวางที่นี่



ผลการวิเคราะห์

0%

คะแนน

{% endblock %}

posture_analysis.py

import numpy as np

```

import mediapipe as mp
import math as m

def calculate_angles(pose_landmarks):

    def check_symmetry(left_point, right_point, threshold=0.05):
        return abs(left_point.y - right_point.y) < threshold

    def findDistance(x1, y1, x2, y2):
        dist = m.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
        return dist

    def check_camera_alignment(l_shldr_x, l_shldr_y, r_shldr_x, r_shldr_y):
        offset = findDistance(l_shldr_x, l_shldr_y, r_shldr_x, r_shldr_y)
        if offset < 100: # ค่า threshold
            return True, f"Camera Aligned: {offset:.1f}"
        else:
            return False, f"Camera Not Aligned: {offset:.1f}"

    def calculate_angle(a, b, c):
        """
        คำนวณมุมระหว่างจุด a, b, c โดยใช้เวกเตอร์
        - จุด a, b, c อยู่ในรูป (x, y) และค่า y จะถูกปรับให้พิกัดเป็น Cartesian (แกน y เพิ่มขึ้นด้านบน)
        """
        a = (a[0], 1.0 - a[1])
        b = (b[0], 1.0 - b[1])
        c = (c[0], 1.0 - c[1])

        ba = np.array([a[0] - b[0], a[1] - b[1]])
        bc = np.array([c[0] - b[0], c[1] - b[1]])

        dot_product = np.dot(ba, bc)
        magnitude_ba = np.linalg.norm(ba)
        magnitude_bc = np.linalg.norm(bc)
        cosine_angle = np.clip(dot_product / (magnitude_ba * magnitude_bc), -1.0, 1.0)

        angle = np.degrees(np.arccos(cosine_angle))
        return round(angle, 2)

    # ตั้ง Keypoints
    LHIP = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_HIP]
    RHIP = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.RIGHT_HIP]
    LKNEE = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_KNEE]
    LANKLE = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_ANKLE]
    RANKLE = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.RIGHT_ANKLE]
    LSHOULDER = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_SHOULDER]
    LELBOW = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_ELBOW]
    LEAR = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.LEFT_EAR]
    REAR = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.RIGHT_EAR]
    RSHOULDER = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.RIGHT_SHOULDER]
    NOSE = pose_landmarks.landmark[mp.solutions.pose.PoseLandmark.NOSE]

    if not pose_landmarks or not pose_landmarks.landmark:
        raise ValueError("Pose landmarks are empty or invalid.")

    if LANKLE.visibility < 0.5 or RANKLE.visibility < 0.5:
        is_visible = "Feet Not Properly Placed"

    aligned, message = check_camera_alignment(LSHOULDER.x, LSHOULDER.y, RSHOULDER.x, RSHOULDER.y)
    shoulder_symmetry = check_symmetry(LSHOULDER, RSHOULDER)
    hip_symmetry = check_symmetry(LHIP, RHIP)

    # คำนวณมุม
    hip_angle = calculate_angle(
        (LHIP.x, LHIP.y),
        (LKNEE.x, LKNEE.y),
        (LANKLE.x, LANKLE.y)
    )
    back_angle = calculate_angle(
        (LSHOULDER.x, LSHOULDER.y),
        (LHIP.x, LHIP.y),
        (LKNEE.x, LKNEE.y)
    )
    neck_angle = calculate_angle(
        (LEAR.x, LEAR.y),
        (LSHOULDER.x, LSHOULDER.y),
        (LHIP.x, LHIP.y)
    )
    shoulder_angle = calculate_angle(
        (LELBOW.x, LELBOW.y),
        (LSHOULDER.x, LSHOULDER.y),
        (LEAR.x, LEAR.y)
    )
    head_tilt_angle = calculate_angle(
        (NOSE.x, NOSE.y), # จมูก
        (LEAR.x, LEAR.y), # หูซ้าย
        (REAR.x, REAR.y)
    )
    knee_angle = calculate_angle(
        (LHIP.x, LHIP.y),
        (LKNEE.x, LKNEE.y),
        (LANKLE.x, LANKLE.y)
    )

    return {

```

```

        "hip_angle": float(hip_angle),
        "back_angle": float(back_angle),
        "neck_angle": float(neck_angle),
        "shoulder_angle": float(shoulder_angle),
        "head_tilt_angle": float(head_tilt_angle),
        "knee_angle": float(knee_angle),
        "camera_aligned": bool(aligned),
        "shoulder_symmetry": bool(shoulder_symmetry),
        "hip_symmetry": bool(hip_symmetry),
        "camera_alignment_message": str(message)
    }

def calculate_score(angles):
    # กำหนดคะแนนเริ่มต้น
    score = 100
    feedback = []

    # มุมสะโพก (Hip Angle): ควรอยู่ระหว่าง 90° - 120°
    hip_angle = angles['hip_angle']
    if not (90 <= hip_angle <= 120):
        error = abs(hip_angle - 105) # ค่าอ้างอิงที่เหมาะสมคือ 105°
        feedback.append(f"ข้อผิดพลาดมุมสะโพก: {error:.1f}°")
        score -= min(15, error * 0.75)
        if hip_angle < 90:
            feedback.append("มุมสะโพกต่ำเกินไป ลองยกสะโพกขึ้นเล็กน้อย.")
        elif hip_angle > 120:
            feedback.append("มุมสะโพกสูงเกินไป ลองลดสะโพกลงเพื่อให้ท่าทางถูกต้อง.")

    # มุมหลัง (Back Angle): ควรอยู่ระหว่าง 160° - 180°
    back_angle = angles['back_angle']
    if not (160 <= back_angle <= 180):
        error = abs(back_angle - 170) # ค่าอ้างอิงที่เหมาะสมคือ 170°
        feedback.append(f"ข้อผิดพลาดมุมหลัง: {error:.1f}°")
        score -= min(15, error * 0.75)
        if back_angle < 160:
            feedback.append("มุมหลังต่ำเกินไป ลองนั่งตัวตรงขึ้น.")
        elif back_angle > 180:
            feedback.append("มุมหลังสูงเกินไป ลองผ่อนคลายและหลีกเลี่ยงการโค้งหลังมากเกินไป.")

    # มุมคอ (Neck Angle): ควรอยู่ระหว่าง 160° - 180°
    neck_angle = angles['neck_angle']
    if not (160 <= neck_angle <= 180):
        error = abs(neck_angle - 170) # ค่าอ้างอิงที่เหมาะสมคือ 170°
        feedback.append(f"ข้อผิดพลาดมุมคอ: {error:.1f}°")
        score -= min(15, error * 0.75)
        if neck_angle < 160:
            feedback.append("มุมคอต่ำเกินไป ลองยกศีรษะให้ตรงกับกระดูกสันหลัง.")
        elif neck_angle > 180:
            feedback.append("มุมคอสูงเกินไป ลองลดศีรษะลงเล็กน้อยเพื่อหลีกเลี่ยงความตึงเครียด.")

    # มุมการเอียงศีรษะ (Head Tilt Angle): ควรอยู่ระหว่าง 85° - 95°
    head_tilt = angles['head_tilt_angle']
    if not (85 <= head_tilt <= 95):
        error = abs(head_tilt - 90) # ค่าอ้างอิงที่เหมาะสมคือ 90°
        feedback.append(f"ข้อผิดพลาดมุมการเอียงศีรษะ: {error:.1f}°")
        score -= min(15, error * 0.75)
        if head_tilt < 85:
            feedback.append("มุมการเอียงศีรษะต่ำเกินไป ลองยกศีรษะขึ้นเพื่อหลีกเลี่ยงการห่อไหล่.")
        elif head_tilt > 95:
            feedback.append("มุมการเอียงศีรษะสูงเกินไป ลองลดศีรษะลงเพื่อหลีกเลี่ยงการตึงของคอ.")

    # ตรวจสอบว่าคะแนนต่ำกว่า 0 หรือไม่
    score = max(score, 0)

    # เพิ่มข้อเสนอแนะตามคะแนนที่ได้
    if score == 100:
        feedback.append("ท่าทางสมบูรณ์แบบ! ต่อไปทำดีแบบนี้เรื่อย ๆ.")
    elif score >= 80:
        feedback.append("ท่าทางดีอยู่แล้ว แต่ยังมีปรับปรุงเล็กน้อยที่ต้องทำ.")
    elif score >= 60:
        feedback.append("ท่าทางพอใช้ได้ แต่ควรปรับปรุงเพิ่มเติม.")
    else:
        feedback.append("ท่าทางต้องการการปรับปรุงมากเพื่อสุขภาพที่ดีขึ้น.")

    return score, feedback

```